# PYTHON ACTIVITY-BASED LEARNING

## 10 Case Studies with Algorithms, Flowcharts and Source Code

Prepared in the same case-study structure as the supplied sample PDF: problem statement, objectives, assumptions, algorithm, pseudocode, flowchart, Python source code, test cases, sample output and conclusion.

Note: All rates, thresholds and policy rules are illustrative learning assumptions. Replace them with values specified by your instructor.

# 1. University Scholarship Allocation

## 1. Problem Statement

Evaluate applicants using aggregate marks, subject-wise minimum marks, attendance and family-income criteria. Rank eligible applicants and resolve ties by predefined rules.

## 2. Objectives

• Check subject minimums and aggregate percentage.

• Check attendance and annual family income.

• Separate eligible and ineligible applicants.

• Rank eligible applicants using tie-break rules.

## 3. Assumptions

• Subjects: Mathematics, Physics and Computer Science; each must be at least 50.

• Aggregate must be at least 70%, attendance at least 75%, and annual income at most ■2,50,000.

• Tie-break: higher aggregate, then higher Mathematics mark, then alphabetical name.

## 4. Algorithm

1. Store applicant records and eligibility limits.

2. Calculate each applicant's aggregate.

3. Check all eligibility conditions.

4. Separate eligible and ineligible applicants.

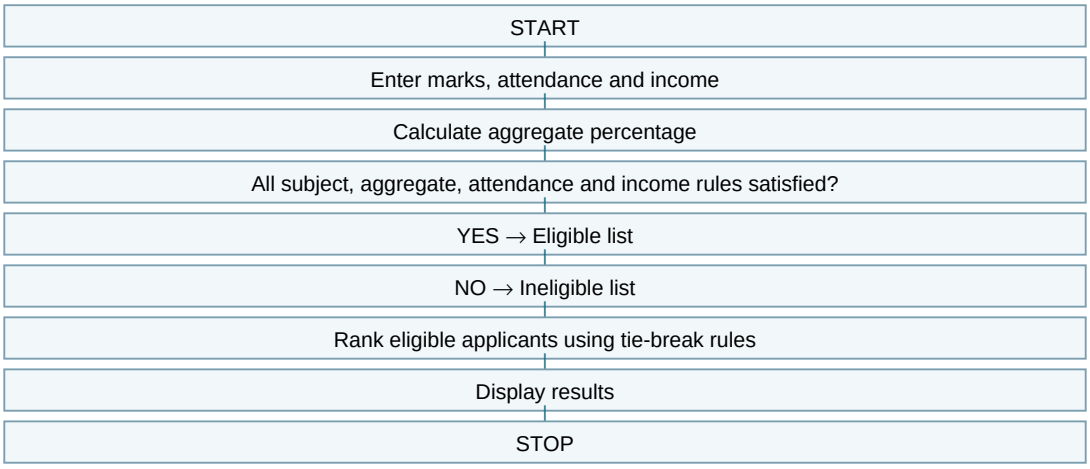
5. Sort eligible applicants by the tie-break rules.

6. Display both lists.

## 5. Pseudocode

```
START
Set minimum marks = 50, aggregate = 70%, attendance = 75%, income limit = 250000
FOR each applicant: calculate aggregate
IF every condition passes: add to eligible list
ELSE: add to ineligible list
Sort eligible by aggregate, Mathematics, then name
Display results
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
applicants = [
    ("Anil", 82, 76, 88, 90, 180000),
    ("Bhavya", 70, 68, 72, 80, 220000),
    ("Charan", 74, 70, 76, 78, 240000),
    ("Esha", 69, 72, 75, 90, 180000)
]
eligible, ineligible = [], []
for name, m, p, c, attendance, income in applicants:
    avg = (m + p + c) / 3
    if min(m, p, c) >= 50 and avg >= 70 and attendance >= 75 and income <= 250000:
        eligible.append((name, m, avg))
    else:
        ineligible.append((name, round(avg, 2)))
eligible.sort(key=lambda x: (-x[2], -x[1], x[0]))
print("ELIGIBLE")
for rank, (name, maths, avg) in enumerate(eligible, 1):
```

```
    print(rank, name, round(avg, 2), "%")
print("INELIGIBLE")
for item in ineligible:
    print(item)
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| Anil: 82,76,88; attendance 90; income 180000 | 82% average | Eligible |
| Bhavya: 70,68,72; attendance 80; income 220000 | 70% average | Eligible |
| Esha: 69,72,75 | 72% average | Ineligible: average below 70? |

## 9. Sample Output / Result

```
Example output: Anil ranks first with 82.0%; Charan follows with 73.33%; Bhavya has 70.0%. Esha is ineligible because her aggregate is be
```

## 10. Conclusion

This program applies multiple scholarship conditions, separates applicants and ranks those who qualify. All thresholds are illustrative.

# 2. Progressive Electricity Billing

## 1. Problem Statement

Calculate household bills using consumption slabs, fixed charges, surcharges and rebates. Compare bills over several months and flag unusual increases.

## 2. Objectives

• Calculate slab-wise energy charges.

• Add fixed charge and surcharge.

• Apply a qualifying rebate.

• Flag a large month-to-month increase.

## 3. Assumptions

• First 100 units: ■2/unit; next 100: ■3/unit; above 200: ■5/unit.

• Fixed charge ■50; surcharge 5% of energy charge; rebate ■30 when use is at most 100 units.

• Increase greater than 30% over previous bill is unusual.

## 4. Algorithm

1. Read monthly unit usage.

2. Calculate slab charges.

3. Add fixed charge and surcharge.

4. Apply rebate if eligible.

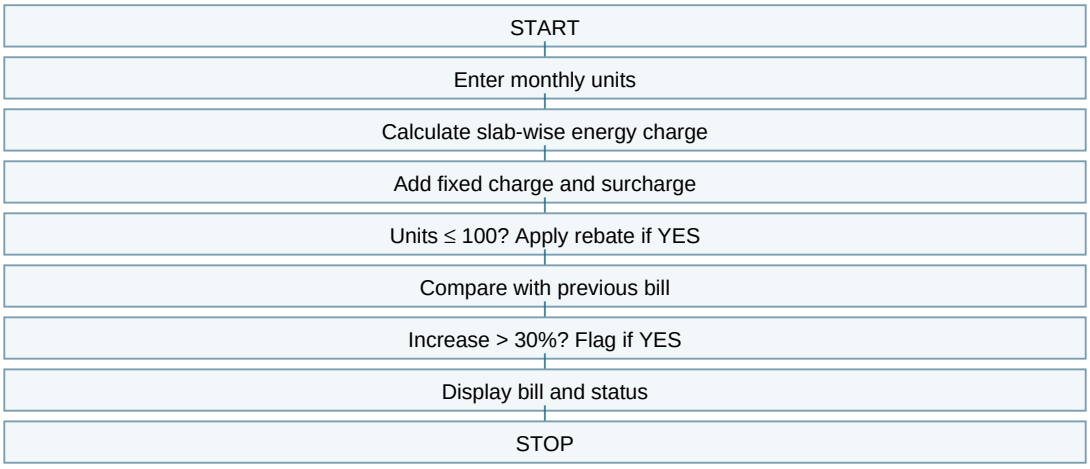
5. Compare bills and flag increases over 30%.

6. Display monthly bills.

## 5. Pseudocode

```
START
Read monthly units
FOR each month: calculate slab-wise energy charges
Add fixed charge and 5% surcharge
IF units <= 100: subtract ■30 rebate
IF bill rises >30% from previous bill: flag unusual
Display bill
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
monthly_units = [90, 110, 180, 260]
previous_bill = None
for month, units in enumerate(monthly_units, 1):
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 200 + (units - 100) * 3
    else:
        energy = 500 + (units - 200) * 5
    bill = energy + 50 + energy * 0.05
    if units <= 100:
        bill -= 30
    status = "Normal"
    if previous_bill is not None and bill > previous_bill * 1.30:
        status = "UNUSUAL INCREASE"
    print("Month", month, "Units:", units,
```

```
        "Bill: ■", round(bill, 2), status)
    previous_bill = bill
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| 90 units | ■208.00 | Rebate applied |
| 110 units | ■365.00 | Normal |
| 260 units | ■1,050.00 | Compare with previous month |

## 9. Sample Output / Result

```
Month 1: 90 units, ■208.00. Month 2: 110 units, ■365.00. Month 3: 180 units, ■585.00. Month 4: 260 units, ■1,050.00 (unusual increase).
```

## 10. Conclusion

The program demonstrates slab billing, fixed charges, surcharges, rebates and anomaly checking.

# 3. Hospital Billing and Patient Classification

## 1. Problem Statement

Calculate consultation, diagnostic and treatment charges, apply conditional concessions, validate charges and classify the final bill.

## 2. Objectives

• Calculate total charges.

• Apply eligible concession.

• Reject negative charges.

• Classify final amount and display itemized bill.

## 3. Assumptions

• Charges cannot be negative.

• Senior citizens aged 60+ receive 10% off when subtotal is at least ■5,000.

• Final amount below ■2,000 = Basic; ■2,000–■7,999 = Standard; ■8,000+ = High-cost.

## 4. Algorithm

1. Read charges and age.

2. Reject negative charges.

3. Calculate subtotal.

4. Apply concession if eligible.

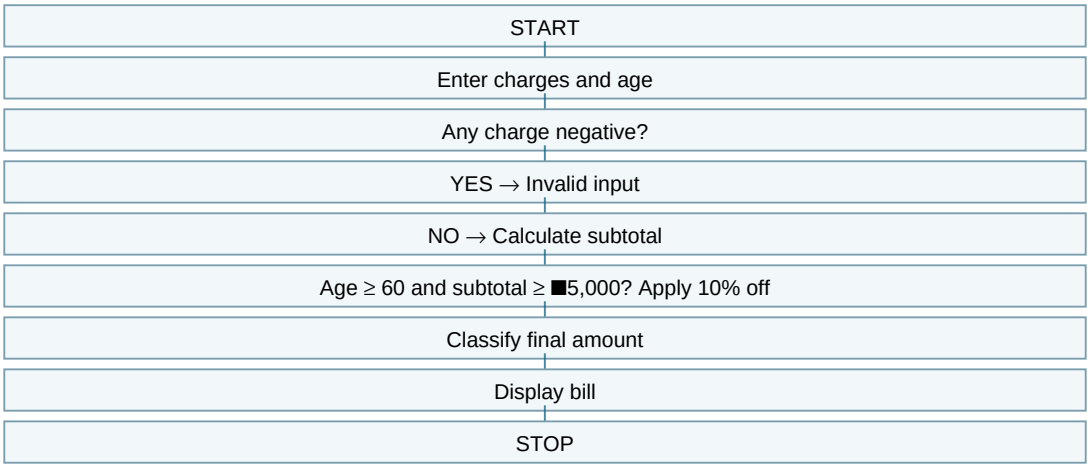
5. Classify final bill amount.

6. Display results.

## 5. Pseudocode

```
START
Read consultation, diagnostic, treatment charges and age
IF any charge is negative: show invalid input
ELSE calculate subtotal
IF subtotal >= 5000 AND age >= 60: apply 10% discount
Classify final amount
Display itemized bill
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
consultation = float(input("Consultation charge: ■"))
diagnostic = float(input("Diagnostic charge: ■"))
treatment = float(input("Treatment charge: ■"))
age = int(input("Patient age: "))
if min(consultation, diagnostic, treatment) < 0:
    print("Invalid bill: charges cannot be negative")
else:
    subtotal = consultation + diagnostic + treatment
    discount = subtotal * 0.10 if subtotal >= 5000 and age >= 60 else 0
    final = subtotal - discount
    if final < 2000:
        category = "Basic"
    elif final < 8000:
        category = "Standard"
    else:
        category = "High-cost"
    print("Subtotal: ■", round(subtotal, 2))
    print("Concession: ■", round(discount, 2))
```

```
print("Final bill: ■", round(final, 2))
print("Category:", category)
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| ■1,000 + ■500 + ■2,000; age 30 | ■3,500 | Standard |
| ■2,000 + ■1,000 + ■3,000; age 65 | ■5,400 | 10% concession |
| Negative charge | Invalid input | Rejected |

## 9. Sample Output / Result

```
Example: subtotal ■6,000; senior citizen concession ■600; final bill ■5,400; category Standard.
```

## 10. Conclusion

The program demonstrates validation, conditional concessions and patient payment classification.

# 4. Employee Payroll and Deduction Analysis

## 1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime and incentives; apply progressive deductions and print a salary statement.

## 2. Objectives

• Calculate allowances and overtime.

• Calculate gross salary.

• Apply progressive deductions.

• Display net salary in a formatted statement.

## 3. Assumptions

• HRA = 20% of basic; other allowance = 10% of basic; overtime = ■200/hour.

• Illustrative deduction: 0% up to ■20,000; 5% on the next ■20,000; 10% above ■40,000.

## 4. Algorithm

1. Read basic pay, overtime hours and incentive.

2. Calculate allowances and overtime pay.

3. Calculate gross salary.

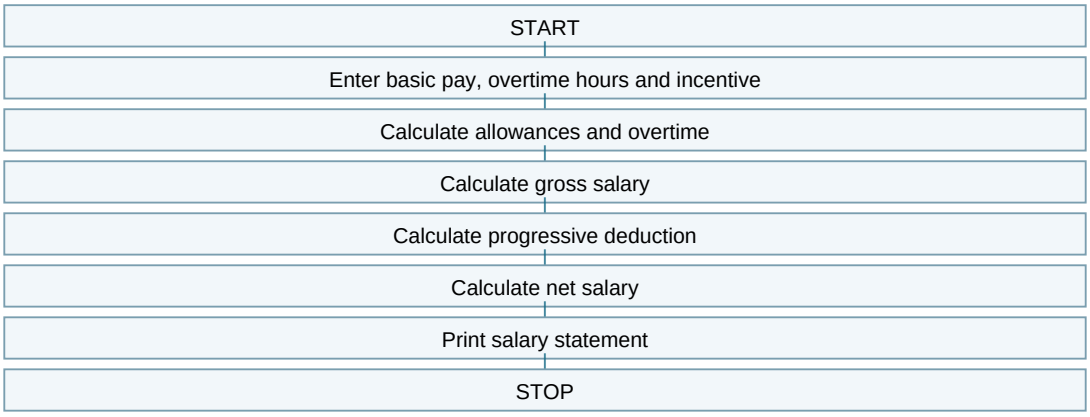
4. Calculate progressive deduction.

5. Calculate net salary and print statement.

## 5. Pseudocode

```
START
Read basic pay, overtime hours and incentive
Calculate HRA, other allowance and overtime
Gross = basic + allowances + overtime + incentive
Apply progressive deduction bands
Net = gross - deduction
Print salary statement
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
basic = float(input("Basic pay: ■"))
hours = float(input("Overtime hours: "))
incentive = float(input("Incentive: ■"))
if min(basic, hours, incentive) < 0:
    print("Invalid input")
else:
    hra = basic * 0.20
    other = basic * 0.10
    overtime = hours * 200
    gross = basic + hra + other + overtime + incentive
    if gross <= 20000:
        deduction = 0
    elif gross <= 40000:
        deduction = (gross - 20000) * 0.05
    else:
        deduction = 1000 + (gross - 40000) * 0.10
    net = gross - deduction
    print("\n--- SALARY STATEMENT ---")
    print("Basic: ■", basic, "HRA: ■", hra)
    print("Other allowance: ■", other)
    print("Overtime: ■", overtime, "Incentive: ■", incentive)
    print("Gross: ■", round(gross, 2))
    print("Deduction: ■", round(deduction, 2))
```

```
print("Net salary: ■", round(net, 2))
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| Basic ■20,000; OT 5 hours; incentive ■1,000 | Gross ■28,000 | Deduction ■400 |
| Basic ■30,000; OT 10 hours; incentive ■2,000 | Gross ■43,000 | Deduction ■2,300 |

## 9. Sample Output / Result

```
For basic ■20,000, 5 overtime hours and ■1,000 incentive: gross ■28,000; deduction ■400; net salary ■27,600.
```

## 10. Conclusion

The program demonstrates payroll calculations, progressive deductions and formatted output.

# 5. Multi-Plan Telecommunications Cost Optimization

## 1. Problem Statement

Compare mobile plans against data usage, validity and budget, calculate excess charges and select the cheapest eligible plan.

## 2. Objectives

• Store plan prices, data limits and validity.

• Calculate excess data charges.

• Check validity and budget.

• Choose the lowest-cost eligible plan.

## 3. Assumptions

• Plan A: ■199, 2 GB/day, 28 days; Plan B: ■299, 3 GB/day, 28 days; Plan C: ■399, 2 GB/day, 56 days.

• Excess data costs ■20/GB. Included data = daily limit × validity.

## 4. Algorithm

1. Read required data, validity and budget.

2. Calculate each plan's included data and excess cost.

3. Calculate total estimated cost.

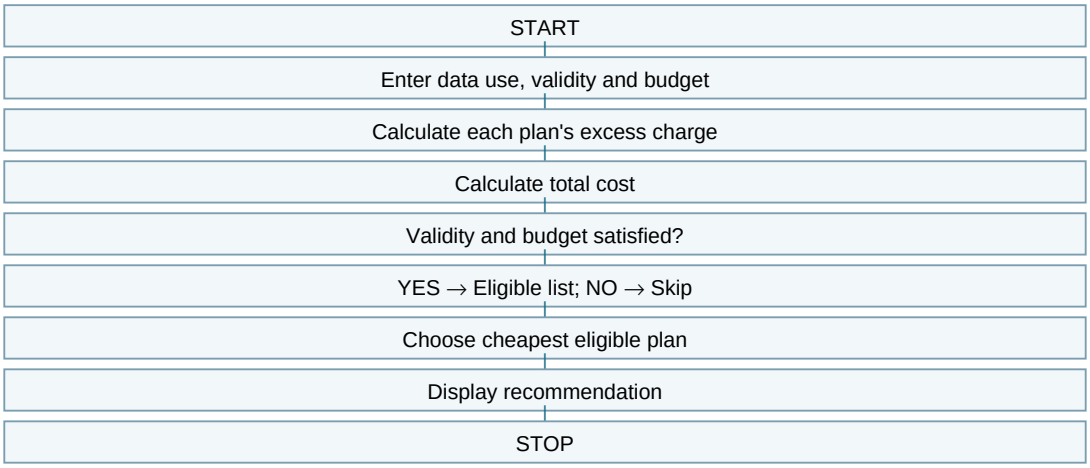
4. Keep plans that meet validity and budget.

5. Select the lowest-cost eligible plan.

## 5. Pseudocode

```
START
Read data use, required validity and budget
FOR each plan: calculate excess data and total cost
IF validity is enough AND cost <= budget: keep plan
Select lowest-cost eligible plan
IF none: report no suitable plan
Display comparison
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
plans = [
    ("Plan A", 199, 2, 28),
    ("Plan B", 299, 3, 28),
    ("Plan C", 399, 2, 56)
]
usage = float(input("Total data required (GB): "))
days_needed = int(input("Validity required (days): "))
budget = float(input("Budget: ■"))
eligible = []
for name, price, daily, days in plans:
    included = daily * days
    excess = max(0, usage - included)
    total = price + excess * 20
    print(name, "cost: ■", round(total, 2))
    if days >= days_needed and total <= budget:
        eligible.append((total, name))
if eligible:
    eligible.sort()
    print("Best plan:", eligible[0][1], "■", eligible[0][0])
else:
    print("No plan meets all requirements")
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| Usage 30 GB, 28 days, budget ■300 | Plan A ■199 | Plan A |
| Usage 70 GB, 56 days, budget ■500 | Plan C ■399 | Plan C |
| Required validity 60 days | No eligible plan | No plan meets constraint |

## 9. Sample Output / Result

```
The program prints each plan's estimated cost and recommends the cheapest plan meeting data, validity and budget requirements.
```

## 10. Conclusion

The program compares plans with common cost rules and selects the most economical eligible plan.

# 6. Railway Fare and Reservation Calculator

## 1. Problem Statement

Calculate fares for multiple passengers using distance, travel class, age concessions, reservation fees and cancellation penalties.

## 2. Objectives

• Calculate fare by distance and class.

• Apply age concessions.

• Add reservation fees.

• Calculate cancellation penalty and refund.

## 3. Assumptions

• Sleeper ■1.50/km and ■40 reservation fee; AC ■3/km and ■100 reservation fee.

• Age 5–11 gets 50% off base fare; age 60+ gets 20% off base fare.

• Cancellation penalty = 25% of discounted base fare; reservation fee is not discounted.

## 4. Algorithm

1. Read passenger details.

2. Validate distance and class.

3. Calculate base fare and age concession.

4. Add reservation fee.

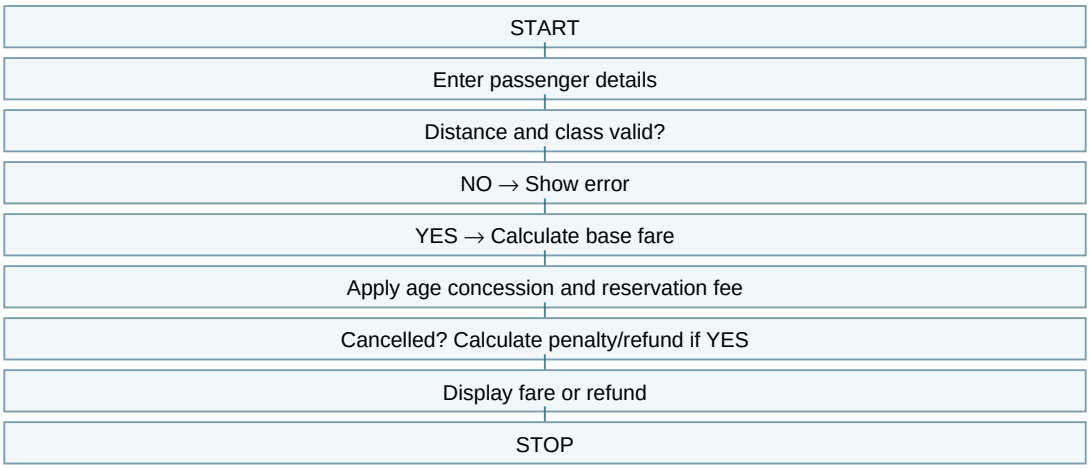
5. If cancelled, calculate penalty and refund.

6. Display passenger-wise result.

## 5. Pseudocode

```
START
Read passenger age, distance, class and cancellation status
Validate inputs
Select class rate and reservation fee
Calculate base fare and age concession
Add reservation fee
IF cancelled: calculate penalty and refund
Display result
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
passengers = [
    ("Ravi", 35, 300, "Sleeper", False),
    ("Meena", 65, 300, "AC", False),
    ("Kiran", 10, 150, "Sleeper", True)
]
for name, age, distance, cls, cancelled in passengers:
    if distance <= 0 or cls not in ("Sleeper", "AC"):
        print(name, "Invalid details")
        continue
    if cls == "Sleeper":
        rate, reservation = 1.5, 40
    else:
        rate, reservation = 3.0, 100
    base = distance * rate
    if 5 <= age <= 11:
        base *= 0.50
    elif age >= 60:
```

```
    base *= 0.80
total = base + reservation
if cancelled:
    penalty = base * 0.25
    print(name, "Penalty: ■", round(penalty, 2),
          "Refund: ■", round(total - penalty, 2))
else:
    print(name, "Fare: ■", round(base, 2),
          "Reservation: ■", reservation,
          "Total: ■", round(total, 2))
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| Adult, 300 km, Sleeper | ■490 total | No concession |
| Senior, 300 km, AC | ■820 total | 20% fare concession |
| Child, 150 km, Sleeper, cancelled | Penalty ■28.13 | Refund ■236.88 |

## 9. Sample Output / Result

```
Adult Sleeper example: 300 × ■1.50 = ■450; plus ■40 reservation = ■490 total.
```

## 10. Conclusion

The program combines distance, class, age and cancellation rules to calculate passenger-wise railway costs.

# 7. Retail Billing with Multiple Discount Rules

## 1. Problem Statement

Prepare an itemized invoice for products, quantities, category-specific taxes, threshold discounts and a maximum discount limit.

## 2. Objectives

• Calculate line totals and subtotal.

• Apply discount threshold and cap.

• Calculate category taxes.

• Print the final invoice.

## 3. Assumptions

• Electronics tax 18%, clothing 5%, books 0%.

• A 10% discount applies to subtotal ≥ ■5,000, capped at ■1,000.

• Tax is calculated on each item's amount after proportional discount.

## 4. Algorithm

1. Store products, category, price and quantity.

2. Calculate subtotal.

3. Calculate discount and apply cap.

4. Allocate discount proportionally.

5. Calculate category tax.

6. Print itemized invoice and grand total.

## 5. Pseudocode

```
START
Read/store product, category, price and quantity
Calculate subtotal
IF subtotal >= 5000: discount = min(10% subtotal, 1000)
Allocate discount proportionally
Calculate category taxes
Grand total = subtotal - discount + taxes
Print invoice
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
items = [
    ("Headphones", "electronics", 2500, 1),
    ("T-Shirt", "clothing", 800, 2),
    ("Notebook", "books", 100, 3)
]
rates = {"electronics": 0.18, "clothing": 0.05, "books": 0.0}
subtotal = sum(price * qty for name, cat, price, qty in items)
discount = min(subtotal * 0.10, 1000) if subtotal >= 5000 else 0
discount_rate = discount / subtotal if subtotal else 0
tax_total = 0
print("ITEMIZED INVOICE")
for name, cat, price, qty in items:
    line = price * qty
    discounted = line * (1 - discount_rate)
    tax = discounted * rates[cat]
    tax_total += tax
    print(name, "Qty:", qty, "Amount: ■", line, "Tax: ■", round(tax, 2))
```

```
print("Subtotal: ■", subtotal)
print("Discount: ■", round(discount, 2))
print("Tax: ■", round(tax_total, 2))
print("Grand total: ■", round(subtotal - discount + tax_total, 2))
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| Subtotal ■5,000 | ■500 discount | Threshold applied |
| Subtotal ■15,000 | ■1,000 discount | Cap applied |
| Books ■100 × 2 | ■200 | No tax |

## 9. Sample Output / Result

```
The invoice prints each item's quantity, line amount and tax, followed by subtotal, discount, tax total and grand total.
```

## 10. Conclusion

The program demonstrates lists, category-based taxes, threshold discounts and maximum concession limits.

# 8. Manufacturing Cost and Profitability Analysis

## 1. Problem Statement

Calculate manufacturing cost from materials, wastage, labour and machine costs. Compare revenue and profit against a target.

## 2. Objectives

• Calculate material and wastage costs.

• Calculate total production cost and revenue.

• Calculate profit and profit percentage.

• Check the target.

## 3. Assumptions

• Wastage cost is a percentage of material cost.

• Total cost = material + wastage + labour + machine costs.

• Profit percentage = profit / total cost × 100.

## 4. Algorithm

1. Read production quantity and cost inputs.

2. Calculate material and wastage costs.

3. Calculate total cost and sales revenue.

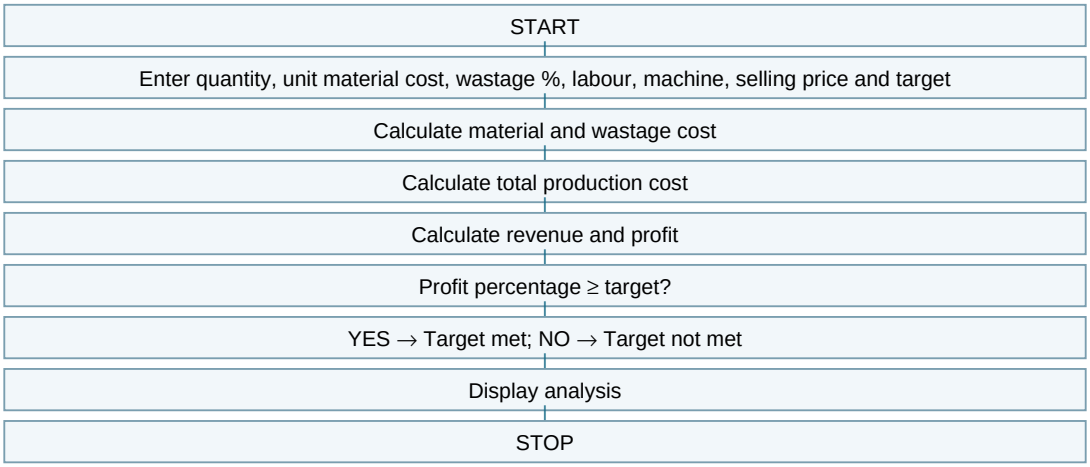
4. Calculate profit and profit percentage.

5. Compare against target and display result.

## 5. Pseudocode

```
START
Read quantity, costs, selling price and target
Calculate material and wastage
Calculate total production cost and revenue
Profit = revenue - total cost
Calculate profit percentage
IF profit percentage >= target: target met
ELSE target not met
Display report
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
quantity = int(input("Units produced: "))
rate = float(input("Material cost per unit: ■"))
waste_pct = float(input("Wastage percentage: "))
labour = float(input("Labour cost: ■"))
machine = float(input("Machine cost: ■"))
selling = float(input("Selling price per unit: ■"))
target = float(input("Target profit percentage: "))
if quantity <= 0 or min(rate, waste_pct, labour, machine, selling, target) < 0:
    print("Invalid input")
else:
    material = quantity * rate
    wastage = material * waste_pct / 100
    cost = material + wastage + labour + machine
    revenue = quantity * selling
    profit = revenue - cost
    pct = profit / cost * 100 if cost else 0
    print("Material: ■", material, "Wastage: ■", round(wastage, 2))
    print("Total cost: ■", round(cost, 2))
```

```
print("Revenue: ■", round(revenue, 2))
print("Profit: ■", round(profit, 2), "Profit %:", round(pct, 2))
print("Target met" if pct >= target else "Target not met")
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| 100 units; rate ■50; wastage 5%; labour ■1,000; machine ■500; sale ■80; target 20% | Cost ■6,750; revenue ■8,000 | Profit ■1,250; target met |
| Same cost; sale ■60 | Revenue ■6,000 | Target not met |

## 9. Sample Output / Result

```
The output shows material, wastage, total cost, revenue, profit, profit percentage and whether the target is met.
```

## 10. Conclusion

The program supports manufacturing cost analysis and a profit-target decision.

# 9. Multi-Currency Travel Budget Analysis

## 1. Problem Statement

Convert travel expenses using supplied exchange rates, add conversion fees, compare spending against a budget and identify the largest expense category.

## 2. Objectives

• Convert foreign expenses to INR.

• Add conversion fees.

• Calculate total spending.

• Compare against budget.

• Identify the largest category.

## 3. Assumptions

• Rates are INR per unit of foreign currency.

• Conversion fee is 2% of converted value.

• Sample rates are illustrative and expenses must be non-negative.

## 4. Algorithm

1. Store expense category, amount and exchange rate.

2. Convert each amount to INR.

3. Add conversion fee.

4. Sum total spending and find largest category.

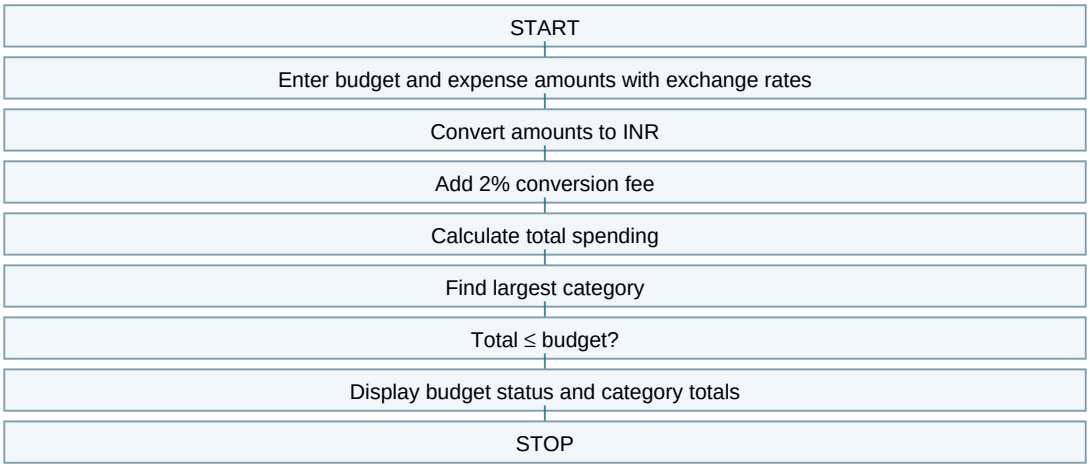
5. Compare total with budget.

6. Display report.

## 5. Pseudocode

```
START
Read budget and expense records
FOR each expense: converted = amount × exchange rate
Add 2% conversion fee
Calculate total and largest category
IF total <= budget: within budget ELSE over budget
Display report
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
expenses = [
    ("Hotel", 200, 83.0),       # USD and INR per USD
    ("Food", 80, 83.0),
    ("Transport", 50, 90.0)    # EUR and INR per EUR
]
budget = float(input("Travel budget in INR: ■"))
totals = {}
for category, amount, rate in expenses:
    converted = amount * rate
    total = converted * 1.02
    totals[category] = total
    print(category, ": ■", round(total, 2))
grand_total = sum(totals.values())
largest = max(totals, key=totals.get)
```

```
print("Total spending: ■", round(grand_total, 2))
print("Largest category:", largest)
if grand_total <= budget:
    print("Within budget; balance ■", round(budget - grand_total, 2))
else:
    print("Over budget by ■", round(grand_total - budget, 2))
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| Budget ■25,000 | Total ■28,294.80 | Over budget |
| Budget ■40,000 | Total ■28,294.80 | Within budget |
| Hotel largest | Hotel | Largest category found |

## 9. Sample Output / Result

```
Hotel ■16,932; Food ■6,772.80; Transport ■4,590. Total ■28,294.80. Hotel is the largest category.
```

## 10. Conclusion

The program combines currency conversion, fees, budget checking and expense-category comparison.

# 10. Campus Event Financial Planning

## 1. Problem Statement

Calculate event costs, participant contributions and sponsorship shortfalls, then find the minimum registration count required to break even.

## 2. Objectives

• Calculate fixed and variable costs.

• Calculate total expected cost and income.

• Find funding shortfall.

• Calculate minimum break-even registrations.

## 3. Assumptions

• Fixed costs include venue and equipment; variable costs are per-person catering, transport and certificates.

• Sponsorship is fixed and registration fee is per participant.

• Break-even is possible only if registration fee exceeds variable cost per participant.

## 4. Algorithm

1. Read fixed costs, per-person costs, sponsorship, fee and expected participants.

2. Calculate fixed and variable costs.

3. Calculate total cost and income.

4. Calculate shortfall.

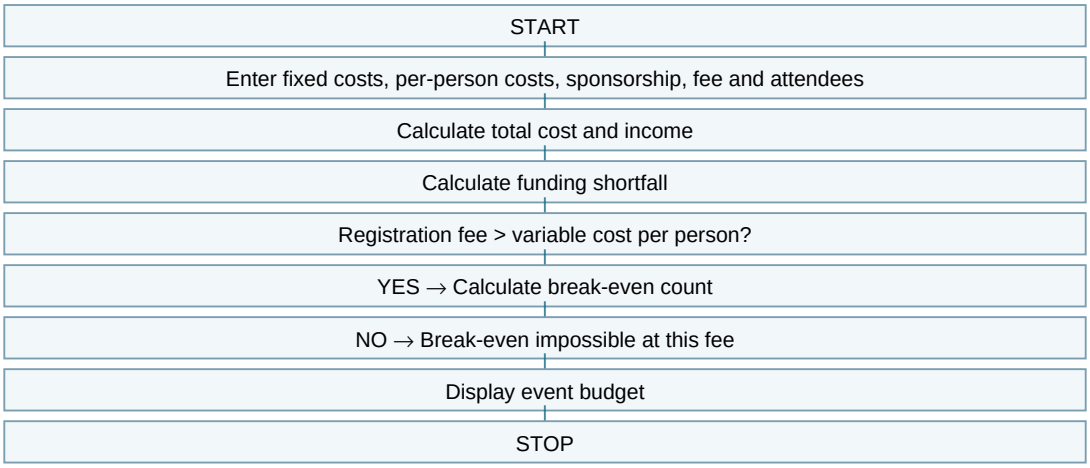
5. Calculate break-even registrations or report impossible.

6. Display the plan.

## 5. Pseudocode

```
START
Read event costs, sponsorship, fee and participant count
Calculate fixed and variable costs
Calculate total cost and income
Shortfall = max(0, cost - income)
IF fee > variable cost: calculate break-even registrations
ELSE break-even impossible at this fee
Display financial plan
STOP
```

## 6. Flowchart




## 7. Python Source Code

```
venue = float(input("Venue cost: ■"))
equipment = float(input("Equipment cost: ■"))
catering = float(input("Catering per person: ■"))
transport = float(input("Transport per person: ■"))
certificate = float(input("Certificate per person: ■"))
sponsor = float(input("Sponsorship: ■"))
fee = float(input("Registration fee per person: ■"))
people = int(input("Expected participants: "))
fixed = venue + equipment
variable = catering + transport + certificate
cost = fixed + variable * people
income = sponsor + fee * people
gap = max(0, cost - income)
print("Fixed cost: ■", fixed)
print("Variable cost/person: ■", variable)
```

```
print("Expected total cost: ■", cost)
print("Expected income: ■", income)
print("Funding shortfall: ■", gap)
contribution = fee - variable
if contribution <= 0:
    print("Break-even not possible at this fee")
else:
    remaining = max(0, fixed - sponsor)
    count = math.ceil(remaining / contribution)
    print("Minimum registrations to break even:", count)
```

## 8. Test Cases

| Input / Test | Expected result | Check |
|---|---|---|
| Fixed ■20,000; variable ■150/person; sponsor ■5,000; fee ■300 | Break-even 100 | 100 registrations |
| Fee ■100; variable ■150 | Not possible | Contribution is negative |
| 80 attendees; fixed ■20,000; variable ■150; sponsor ■5,000; fee ■300 | Cost ■32,000; income ■29,000 | Shortfall ■3,000 |

## 9. Sample Output / Result

```
Example: fixed cost ■20,000; variable cost ■150/person; sponsorship ■5,000; fee ■300. Contribution = ■150/person; remaining fixed cost =
```

## 10. Conclusion

The program estimates an event budget, identifies funding gaps and calculates the minimum registrations needed to break even.